2025.08.16

김무진, 오세리, 박진우, 박시은

MAP3K5(ASK1) IC50 활성값 예측 모델 개발

FOM

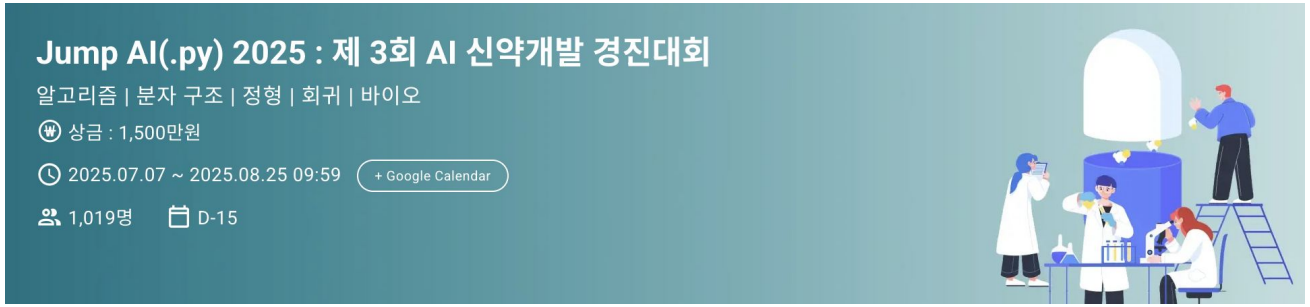
Focus On Data Mining

INDEX

1. 개요
2. 데이터 설명
3. 데이터 전처리
4. 모델링
5. 모델 평가
6. 결론

MAP3K5(ASK1) IC50 활성값 예측 모델 개발

- 한국제약바이오협회 주관 제 3회 AI 신약개발 경진대회 (DACON)



- https://dacon.io/competitions/official/236530/overview/description

01 | 개요

프로젝트 목표

- **MAP3K5(ASK1) ?**
  - 세포 스트레스(산화, 염증 등)에 반응하여 세포 사멸 및 염증반응을 조절하는 신호전달 단백질
  - 세포 성장, 분화, 염증, 세포 사멸 등 다양한 생물학적 과정에 깊이 관여 및 암, 당뇨병, 신경퇴행성 질환 등 여러 질환과 연관

- **IC50 활성값 ?**
  - 어떤 물질(예: 약물, 억제제)이 특정 생물학적·생화학적 기능을 50%만큼 억제하는 데 필요한 농도
  - IC50 값이 낮을수록 해당 물질의 효능(억제력)이 더 높음

02 | 제공 데이터

데이터 설명

ChEMBL, PubChem, CAS 총 세 개의 DB에서 수집한 실험 기반 화합물 정보

- 각 **DB**에 대한 총 세 개의 **csv** 파일 제공

- 출처
  - **PubChem -** 미국 국립 생물 공학정보 센터
  - **ChEMBL -**유럽 바이오 정보 연구소
  - **CAS -** 미국화학회

02 | 제공 데이터

데이터 설명

- **ChEMBL_ASK1(IC50).csv**
  - ChEMBL Database 데이터
  - 논문 기반의 신뢰도 높은 생화학 실험 결과를 수록한 DB
  - ID, 분자명, 분자량(g/mol), 분자 구조 표현(SMILES), 측정 타입, IC50 수치 (nM), pIC50 값, 실험 정보 등 48개의 column 존재

- **Pubchem_ASK1.csv**
  - Pubchem Database 데이터
  - 자동 수집 + 사용자 제출 기반이라 데이터 품질 편차 존재 가능성
  - IC50 수치 (μM), 분자 구조 표현(SMILES), 표적 유전자/단백질, 실험 정보 등 33개의 column 존재

02 | 제공 데이터

데이터 설명

- **ChEMBL_ASK1와 Pubchem_ASK1에서 공통으로 추출할 수 있는 Columns**

Feature	ChEMBL	PubChem	설명
SMILES	Smiles	SMILES	분자의 구조를 표현하는 표준 문자열
IC50 or bioactivity	Standard Value + Standard Type	Activity_Value + Activity_Type	활성 값 (예: IC50, EC50 등)을 통해 모델 타깃 변수 생성
Activity 단위	Standard Units	Activity_Qualifier or 직접 단위 표기 없음	단위 (nM, μM 등) 일치 필요
Target 정보	Target Name, Target ChEMBL ID, Target Organism	Target_Name, Target_Link, Target_Taxonomy_ID	동일한 표적 단백질 정보로 매핑 가능

02 | 제공 데이터

데이터 설명

- **CAS_KPBMA_MAP3K5_IC50s.xlsx**

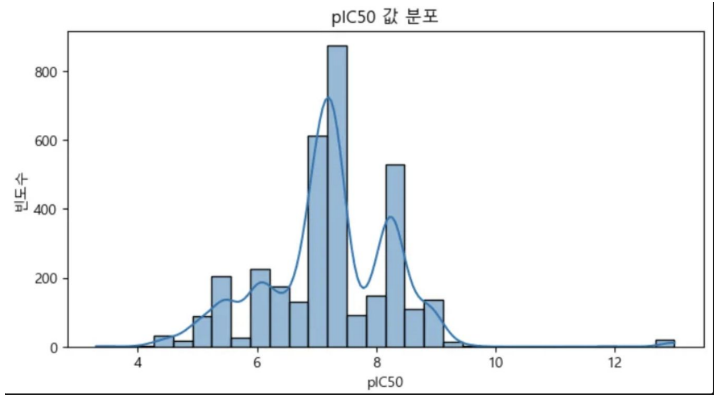
  - KPBMA (한국제약바이오협회) + CAS 데이터를 두 개의 시트에 각각 정리한 Excel 파일

  - 마찬가지로 많은 데이터가 있으나 분자 구조 표현(SMILES) 등의 형식이 다르거나 정제되지 않은 데이터가 많아 학습 시 성능 저하 확인
    - 사용하지 않는 것으로 결정

03 | 제공 데이터

기타 용어

- **Smiles ?**
  - ✔ 화학 분자의 구조를 컴퓨터가 읽을 수 있도록 간단한 ASCII 문자들의 문자열로 표현하는 방법
  - ✔ 원소기호와 -, =, #, @ 기호를 통해 결합 구조와 입체화학 정보를 표현
- **pIC50 ?**
  - ✔ **IC50 값(nM)을 로그 스케일로 변환한 값**
  - ✔ **정규성 향상**: 로그 스케일이 더 정규분포에 가까움
  - ✔ **모델 안정성**: IC50는 범위가 너무 넓고 skewed 함
  - ✔ **화학적 해석 용이**: pIC50는 높을수록 강력한 억제 효과



02 | 제공 데이터

test 데이터

- 앞서 살펴본 파일들에 매우 많은 열들이 있으나,
  - ✔ 예측해야 할 **test.csv**의 컬럼은 **Smile**밖에 없음

test.head()

ID	Smiles
TEST_000	CC(C)n1cnnc1c2cccc(n2)N3C=Cc4ccc(Br)cc4C3=O
TEST_001	C[C@H](CO)n1cnnc1c2cccc(n2)N3C=Cc4ccc(Br)cc4C3=O
TEST_002	C[C@H](CO)n1cnnc1c2cccc(n2)N3C=Cc4cc(Br)ccc4C3=O
TEST_003	CC(C)n1cnnc1c2cccc(n2)N3C=NNC3=O
TEST_004	C[C@H](CO)n1cnnc1c2cccc(n2)N3C=Cc4cc(ccc4C3=O)c5ccc(nc5)C6CC6
TEST_005	CC(C)n1cnnc1c2cccc(n2)N3C=Cc4cc(ccc4C3=O)c5ccc(nc5)C6CC6

- **Smiles**를 중심으로 한 전처리나 모델링 기법에 중점

03 데이터 전처리

기본 제공 전처리

- **Morgan Fingerprint**
  - ✔ Morgan Fingerprint는 분자를 컴퓨터가 인식할 수 있도록 변환한 이진 벡터(0과 1의 배열)로, 분자 내 원자 주변의 구조 정보를 숫자로 표현한
  - ✔ 변환이 되지 않는 유효하지 않은 smiles 필터링

```python
def smiles_to_fingerprint(smiles):
    mol = Chem.MolFromSmiles(smiles)
    if mol is not None:
        fp = AllChem.GetMorganFingerprintAsBitVect(mol, 2, nBits=CFG['NBITS'])
        return np.array(fp)
    else:
        return np.zeros((CFG['NBITS'],))
```

- **IC50 -> pIC50**
  - ✔ 모델 학습에 앞서 정규성 & 안정성 확보

```python
def pIC50_to_IC50(p): return 10 ** (-p)
```

03 데이터 전처리

데이터셋

- **ChEMBL_ASK1(IC50).csv**
  - ✔ Standard Type == IC50 인 열만 필터링
  - ✔ nM 기준의 IC50 수치인 Standard Value 값을 추출
  - ✔ IC50 값을 학습에 용이하도록 로그 변환한 pIC50 값으로 변환

- **Pubchem_ASK1.csv**
  - ✔ IC50값에 1000을 곱하여 단위를 통일 (μM -> nM)
  - ✔ IC50 값을 학습에 용이하도록 로그 변환한 pIC50 값으로 변환

03 데이터 전처리

데이터셋

- **앞선 두 데이터셋을 Merge**
  - smiles를 기준으로 병합 및 drop_duplicates
  - IC50의 값이 결측값이거나 음수이면 제외

- **더 다양한 전처리에 앞서 ChEMBL, Pubchem, Merge 총 세 개의 데이터셋을 기본 제공된 전처리 코드와 기본 파라미터로 설정된 모델을 이용하여 각각 학습**

  -> 가장 성능이 좋은 Merge 데이터셋을 사용하기로 결정

03 데이터 전처리

시행착오

1. 데이터 증강

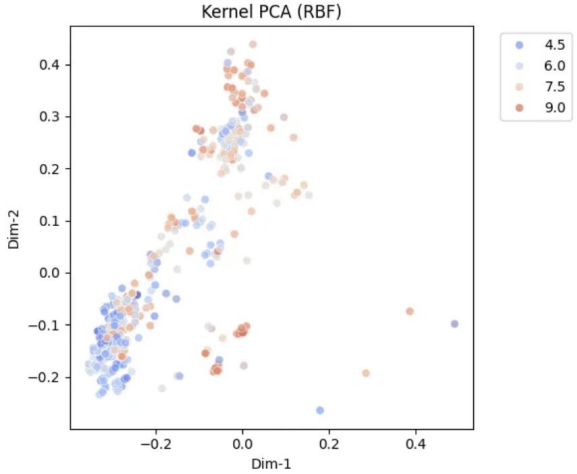
- 화학적으로 같은 분자를 다양한 Smiles 분자 구조로 생성
- 오히려 유효하지 않은 Smiles가 대량으로 생성되어, 모델 성능 대폭 저하

2. 사전 학습 모델

- 수백만 개의 분자(SMILES 또는 그래프)를 기반으로 사전 학습된 모델
- Smiles를 Fingerprint 벡터가 아닌 자연어로 처리하여 임베딩
- 성능 저하로 폐기

3. 차원 축소

- Smiles를 RDKit 라이브러리를 통해 변환한 Fingerprint는 2048차원
- PCA (n_components = 128) 시도
  - 총 분산 : 0.9314606384035566
  - Trustworthiness: 0.876 | Silhouette Score: 0.722



- 준수한 설명력을 보이나 클러스터링이 잘 진행되지 않고, 모델 학습 시 성능 저하

4. K-Fold 교차 검증

- 높은 일반화 성능 기대 (k=3 or 5)
- 학습에 사용된 데이터의 개수 : 1145
  - 너무 적은 데이터로 인해 성능 저하

5. 특성 추출

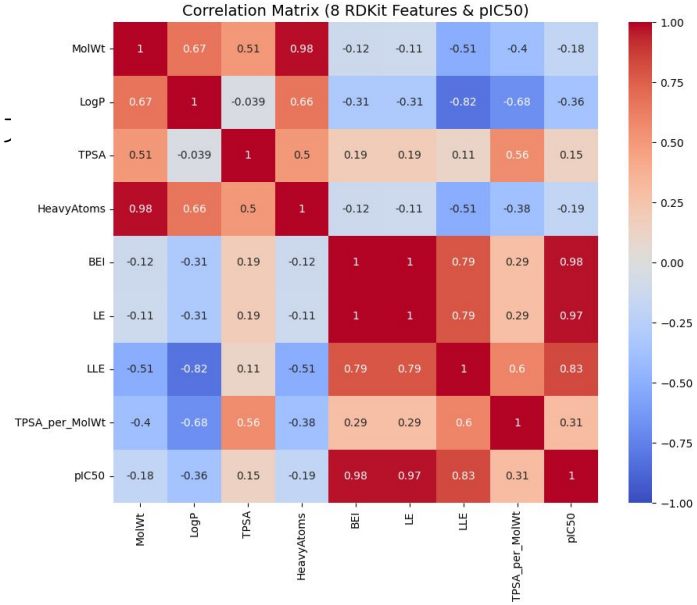
- Smiles를 가지고 해당 분자에 대한 특성을 추출하여 부가적인 입력 feature로써 활용 가능
- 기존에 Smiles로만 학습시킨 모델에 비해 좋은 성능 기대
- 학습에 사용된 파생 특성 (순서대로)
  - 분자량
  - LogP : 지용성/수용성 여부
  - 극성 표면적
  - HeavyAtoms : 무거운 분자의 개수
  - BEI : Binding Efficiency Index
  - LE : Ligand Efficiency
  - LLE : Lipophilic Ligand Efficiency
  - TPSA_per_MolWt : 극성 표면적 / 분자량

```python
def calc_features(smiles, pIC50=7.0):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        return None
    try:
        MolWt = Descriptors.MolWt(mol)
        LogP = Descriptors.MolLogP(mol)
        TPSA = Descriptors.TPSA(mol)
        HeavyAtoms = Descriptors.HeavyAtomCount(mol)
        BEI = pIC50 / (MolWt + 1e-6)
        LLE = pIC50 - LogP
        LE = pIC50 / (HeavyAtoms + 1e-6)
        TPSA_per_MolWt = TPSA / (MolWt + 1e-6)
        return [LE, BEI, LLE, TPSA_per_MolWt]
    except:
        return None
```

5. 특성 추출 – SMILES 기반 RDKit 분자 특성

- **피처 간 상관관계**
  - BEI(0.98), LE(0.97), LLE(0.83) → pIC50과 강한 양의 상관관계
  - BEI ↔ LE (1)
    - 완전 상관관계이므로 둘 중 하나의 값만 사용
  - BEI, LE ↔ LLE (0.79) → 양의 상관 관계
  - LogP ↔ LLE (-0.82) →음의 상관 관계

- **특성 선택**
  - pIC50과 상관 관계가 높은 변수 중 **BEI, LLE** 사용
    - 다중 공선성 문제
      - 트리 기반 모델에서 영향이 적다
  - **TPSA_per_MolWt:** 대부분의 주요 피처와 상관관계 낮음
    - BEI, LE, LLE와 독립적인 정보를 제공 받을 수 있다



6. 데이터 병합 및 중복 제거

- **Pubchem_ASK1.csv**
  - pIC50 = −log10(Activity_Value) 변환
- **ChEMBL_ASK1(IC50).csv**
  - pIC50 값 사용
- 각각의 데이터에서 **SMILES**와 **pIC50**만 추출하여 병합
- 동일한 **SMILES** 중복 제거

```
pub = pd.read_csv("Pubchem_ASK1.csv")
pub = pub[pub['Activity_Value'] > 0].copy()
pub['pIC50'] = -np.log10(pub['Activity_Value'])
pub = pub[['SMILES', 'pIC50']].dropna()

chem = pd.read_csv("ChEMBL_ASK1(IC50).csv", sep=';')
chem = chem[['Smiles', 'pChEMBL Value']].dropna()
chem = chem.rename(columns={'Smiles': 'SMILES', 'pChEMBL Value': 'pIC50'})

df = pd.concat([pub, chem], ignore_index=True).drop_duplicates('SMILES').reset_index(drop=True)
```

7. 결측치 처리 및 데이터 분할

- **결측치 처리**
  - SMILES 파싱 실패 또는 RDKit 계산 불가로 인해 생성되지 않은 파생 피처 제거
  - 테스트 데이터에서 생성 불가 샘플은 평균 pIC50을 사용하여 대체
- **데이터 분할**
  - 모델 검증을 위해 학습/검증 데이터 분리

```
train_df = train_df.dropna().reset_index(drop=True)

X = train_df[['BEI', 'LLE', 'TPSA_per_MolWt']].values
y = train_df['pIC50'].values
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
fallback_ic50_nM = pIC50_to_IC50_M(y.mean()) * 1e9
submission['ASK1_IC50_nM'] = submission['ASK1_IC50_nM'].fillna(fallback_ic50_nM)
```

04 | 모델링

모델 선정

- 후보 모델
  - 화학 물성/분자 특성 데이터는 비선형 관계와 변수 간 복잡한 상호작용이 존재
  - 선형 모델보다 트리 기반 모델이 이러한 복잡성을 잘 포착
  - RandomForest / XGBoost / LightGBM

	RMSE	R^2
RandomForest	0.4167	0.9842
XGBoost	0.4416	0.9823
LightGBM	0.4613	0.9806

- RandomForest가 가장 낮은 RMSE와 높은 R^2을 보여 모델로 선정

04 | 모델링

모델 하이퍼파라미터

- 하이퍼파라미터
  - n_estimators: 생성할 트리의 개수
  - max_depth: 각 트리의 최대 깊이
  - max_features: 각 트리에서 분할 시 고려할 특성(피처)의 개수
  - min_samples_leaf: 리프(Leaf) 노드가 가져야 하는 최소 샘플 수
  - min_samples_split: 노드를 분할하기 위해 필요한 최소 샘플 수
  - random_state: 난수 생성 시드 값
  - n_jobs: 사용할 CPU 코어 수

```
model = RandomForestRegressor(
    n_estimators=314,
    max_depth=25,
    max_features=None,
    min_samples_leaf=2,
    min_samples_split=4,
    random_state=42,
    n_jobs=-1
)
```

05 | 모델 평가

모델 성능

- **RMSE: 0.3986**
- **R^2: 0.9855**

- **데이콘 : 0.61311**
  - **Score : 12위 / 938팀**

• 평가 산식

• **A** = IC50(nM) 단위의 Normalized RMSE 오차, 예측 정확도 측정

$$Normalized\ RMSE = \frac{RMSE}{\max(y) - \min(y)}$$

• **B** = pIC50 변환값을 기준으로 한 예측값과 실제값 간의 **선형 상관관계의 제곱**

$$R^2 = Pearson(pIC_{50}^{true}, pIC_{50}^{pred})^2$$

• **Score** = 0.4 x (1 - min(**A**, 1)) + 0.6 x **B**

$$Score = 0.4 \times (1 - \min(Normalized\ RMSE, 1)) + 0.6 \times R^2$$

12	진우박	진우	0.61311	18	2일 전

06 | 결론

아쉬운 점

- BEI, LLE 값이 PIC50을 기반으로 계산
  - 학습 시 미래 값을 예측하는 형태가 되어 모델 성능에 불리하게 작용

- 화학 분자·약물에 대한 사전 지식 부족
  - 데이터 처리의 세밀함 부족

Q & A

감사합니다